with the organisation's security requirements. Rather than replacing engineering decisions, AI supported the design process through structured technical discussions and evaluation of trade-offs between multiple solutions.

The knowledge integrator. Organisations often possess vast amounts of valuable information, but it is typically distributed across multiple platforms rather than stored in a single repository. Technical documentation may reside in Confluence, project information in Jira, collaboration records in SharePoint, and operational knowledge within internal documentation systems. Although the required information already exists, locating it often consumes considerable time and slows decision-making.

AI can bridge these disconnected knowledge sources by acting as a system integrator. Instead of requiring employees to search multiple applications manually, AI-powered assistants can retrieve relevant information from different enterprise platforms through application programming interfaces (APIs), model context protocol (MCP) servers, and other connectors. This approach allows organisations to build assistants that answer queries using information from multiple internal systems, reducing the effort required to locate technical documents, project records, and operational data.

Natural language interfaces further simplify access to enterprise information. Traditional APIs often require strict request formats and detailed knowledge of system interfaces, whereas AI-powered interfaces allow users to retrieve information through conversational queries. Within secure enterprise environments, this can allow organisations to integrate existing knowledge repositories while keeping sensitive information within internal networks. It can also simplify collaboration and knowledge management.

AI as a force multiplier. Beyond improving access to information, AI can also improve engineering productivity by automating repetitive development activities. Software quality assurance is one area where this capability can have a direct impact. Accessibility testing, for example, has traditionally involved manual evaluation alongside automated tools, with testing teams assessing applications against accessibility guidelines and reporting issues to development teams. This process often involves multiple review cycles and can extend software delivery timelines.

> AS AI BECOMES INCREASINGLY INTEGRATED INTO ENTERPRISE SOFTWARE DEVELOPMENT, THE SKILLS EXPECTED OF DEVELOPERS ARE ALSO EVOLVING.

AI-assisted testing can automate many of these activities. Modern AI systems can generate accessibility test cases, execute evaluations, and identify potential compliance issues before software reaches manual review stages. However, automatically generated outputs should not be accepted without verification. Development teams must compare AI-generated test cases with existing automation frameworks to avoid unnecessary duplication while ensuring that quality assurance processes remain consistent and maintainable. Human oversight remains necessary to validate AI-generated recommendations.

By reducing time spent on routine engineering tasks, AI allows development teams to focus on activities such as system design, feature development, and problem-solving. It can also allow individual developers and small teams to prototype complex applications more quickly, making experimentation and feedback cycles shorter. Although AI can make rapid prototyping easier, deploying production-ready solutions still requires careful integration, validation, and long-term maintenance.

Skills that still matter

As AI becomes increasingly integrated into enterprise software development, the skills expected of developers are also evolving. Rather than measuring productivity through lines of code, completed tickets or story points, organisations are increasingly evaluating developers based on their ability to solve business problems, evaluate alternative approaches, and deliver complete production-ready solutions. Experimenting with AI tools has also become part of software engineering, as different technologies may suit different development tasks.

Developers therefore need structured thinking alongside technical expertise. Understanding the overall system architecture, critically evaluating AI-generated recommendations, and maintaining ownership of the system context remain essential responsibilities that cannot be delegated entirely to AI. Specification-driven development, where requirements and system behaviour are documented before implementation, can also improve AI-assisted workflows by giving models the context needed to generate more relevant and maintainable solutions.

Myths worth busting

The growing adoption of AI has also given rise to several misconceptions about its impact on software engineering. One of the most common assumptions is that AI will eventually replace software developers. In practice, AI is being used primarily